



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

数字逻辑 DIGITAL LOGIC

Instructor: Yuhui BAI

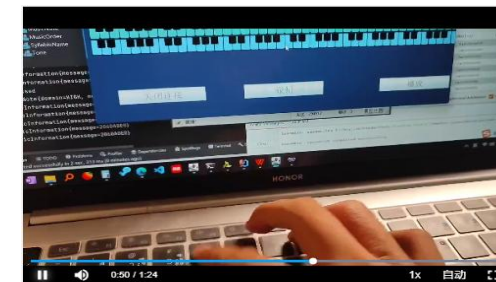
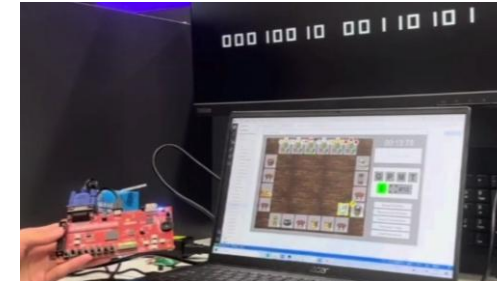
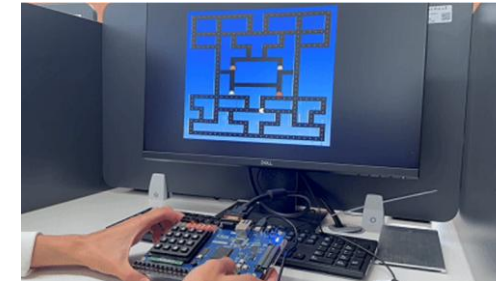


Course Information

- QQ Group: 1070498906
 - password: 207207
- Instructor:
 - Yuhui BAI (baiyh@sustech.edu.cn)
 - Office: 114 College of Engineering South
 - Office hour: Fri. 16:00-18:00 (by appointment)
- Lecture
 - 14:00-15:50 Tue., 207 Lecture Hall #3
- Lab
 - 16:20 -18:10 Tue., Lecture Hall #3 (Yuhui BAI)
 - 10:20 -12:10 Wed., Lecture Hall #3 (Wei WANG)
 - 14:00 -15:50 Wed., Lecture Hall #3 (Wei WANG)

Course Objectives

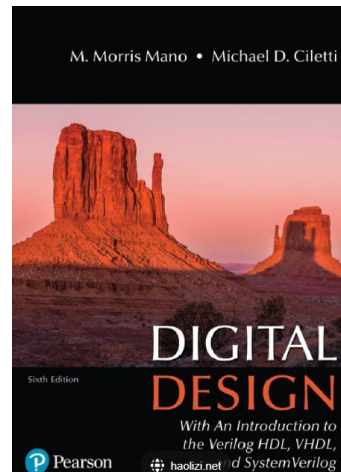
- Understand the fundamentals of digital logic and number systems
- Analyze and design combinational and sequential circuits
- Model, simulate, and verify digital circuits using Verilog HDL
- Implement and integrate digital systems on FPGA platforms



Textbook



- Textbook:
 - Digital Design: With an Introduction to the Verilog HDL, VHDL, and System Verilog by *M. Morris Mano and Michael D. Ciletti*, 6th edition.



- Reference book:
 - Digital Principles and Logic Design by A. Saha and N. Manna.
 - Digital Logic Design by B. Holdsworth and C. Woods

Grading criteria



- Lecture Attendance 5%
 - In-class attendance
- Lab Attendance 5%
 - Lab session attendance
- Lecture Homework 10%
 - 4 assignments on theoretical OJ, 1-week deadline
- Lab Homework 10%
 - Verilog coding assignments on lab OJ, 1-week deadline
- Project 15%
 - Teams of 3, released Week 10
- Final Exam 55%
 - Closed-book written exam
- All assignments/projects this semester will be checked for plagiarism. Violators will be handled according to the undergraduate course plagiarism policy:
 - First offense: A warning and a zero on the work.
 - Second offense: A zero for the entire course.

Course Outline



- Outline:

1. Digital Systems and Binary Numbers

Binary Systems, Conversions, Signed Binary, Codes

2. Boolean Algebra and Logic Gates

Theorems, Boolean Functions, operators, gates

3. Gate-level Minimization

Truth table, K Map, two-level implementations, NAND, NOR

4. Combinational Logic

Combinational circuits, arithmetic logic, mux, de-mux, encoder, decoder

5. Synchronous Sequential Logic

Sequential circuit, Latches, Flip flops, State Machines

6. Registers and Counters

7. Memory and Programmable Logic

RAM, ROM, FPGA

Tentative Schedule



W	Lecture	Lab	
1	Lec1: Course Introduction, Number Systems	Lab1: Environment Setup	
2	Lec2: Boolean Algebra	Lab2: Structural-Based Design, Testbench	
3	Lec3: Gate Level Minimization with Kmap	Lab3: Dataflow Design, Design Flow	HW1 release
4	Lec4: Combinational Logic	Lab4: Behavioral Modeling, 7-Seg Display	HW1 due
5	Holiday	Holiday	
6	Lec5: Standard Components – Decoder	Lab5: Counter, Frequency Divider	
7	Lec5: Standard Components – Mux	Lab6: Flowing Light, Dynamic 7-Seg Display	HW2 release
8	Lec6: Latches and Flip-flops	Lab7: Button Debouncing, Edge Detector	HW2 due
9	Lec7: Sequential Logic Analysis	Lab8: Design Integration	
10	Lec7: Sequential Logic Design	Lab9: Finite State Machine	Project release
11	Lec-extra: UART	Lab10: UART Practice	HW3 released
12	Lec8: Registers	Lab11: Demo	HW3 due
13	Lec8: Counters	Lab12: Demo	
14	Lec9: Two-Level Implementation	Lab13: Lab Review	
15	Lec10: Standard Components – Arithmetic	In-Class Project Test	HW4 released
16	Lec11: Memory and Programmable Logic	Project Defense (Bonus part)	HW4 due

课程贯通培养

- **嵌入式设计大赛**: FPGA创新设计、嵌入式应用 (MCU/MPU/SOC)
- **集创赛**: SoC设计、处理器应用、FPGA应用、AI芯片应用、具身智能应用
- **系统能力大赛**: CPU设计 (龙芯杯)、智能系统 (小米杯)
- 如有疑问可发邮件至: baiyh@sustech.edu.cn



相关竞赛



能力递进: 数字电路 → CPU设计 → 高性能计算机 → 系统实现与应用



嵌赛-FPGA创新设计赛道 (26Fall可参赛)

- **FPGA创新设计赛道报名时间：2026年7月6日-9月22日24点**
 - <http://fpgachina.cn/index.html?page=download>
 - 作品提交时间：2026年11月4日18点
 - 竞赛决赛时间：2026年11月20日-11月22日

主要命题 / 平台方向

AMD | 基础能力 · HLS · FPT国际赛项
从基础设计到AI加速，面向未来的FPGA创新

01 基础能力命题

- Vivado / RTL设计
- 组合与时序电路、FIFO / UART
- ALU、寄存器堆等CPU基础组件



02 HLS优化命题

- Vitis HLS
- SHA-256 / LZ4 / Cholesky
- 目标：正确性 + 时延 + 资源优化



03 FPT 2026 国际赛项

- LLM4HLS Agent
- Attention Acceleration
- 面向AI辅助硬件设计与FPGA加速



安路

- 千兆以太网 / 高速通信
- 图像视频处理
- FPGA+ARM异构SoC
- 自主命题



高速通信 · 图像处理 · 异构SoC

紫光同创

- FPGA智能信号分析与测试
- ADC + FFT / 仪器仪表
- 缺陷检测 / ARM+FPGA 机器视觉



信号处理 · 测试分析 · 机器视觉

高云

- 全景会议相机
- 多摄像头拼接 / 实时视频
- UVC / 网口 / 麦克风阵列扩展



多摄像头 · 实时视频 · 智能人机

易灵思

- RISC-V + FPGA协同
- 人脸识别 / 图像处理
- ADC UI / 多路图像拼接
- 自主命题



开源架构 · 图像处理 · 场景创新

典型能力：
Verilog/RTL > 时序与接口 > 存储与视频 > HLS/硬件加速 > 系统集成

优秀作品画像

1 系统完整

采集 → 处理 → 传输 → 显示形成闭环

2 指标量化

帧率 / 吞吐 / 时延 / 资源占用有数据

3 FPGA参与度高

核心算法真正落在RTL/HLS硬件中

4 工程稳定

多接口联调、长时间运行、现场演示可靠

5 创新突出

AI、RISC-V、异构SoC、高速接口等形成亮点

安路一等奖作品：

UDP + 摄像头 + SDRAM +
FPGA实时图像处理 + HDMI/PC显示

高水平作品常见：

最佳工程奖 / 企业杯 / FPT国际赛项推荐

Lecture 1: Number Systems

- Textbook Chapter 1

Outline

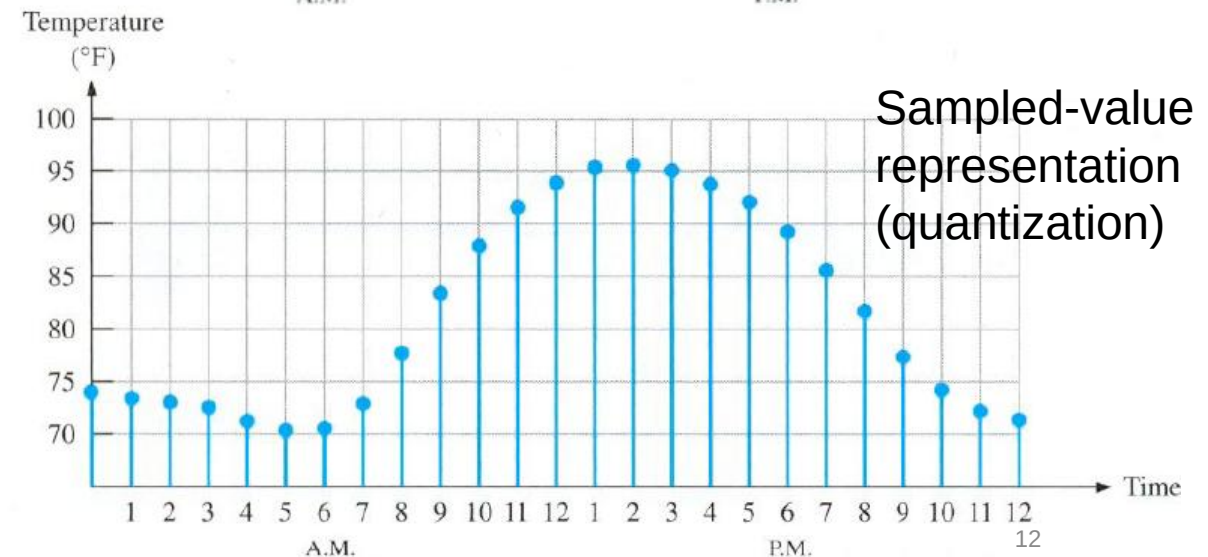
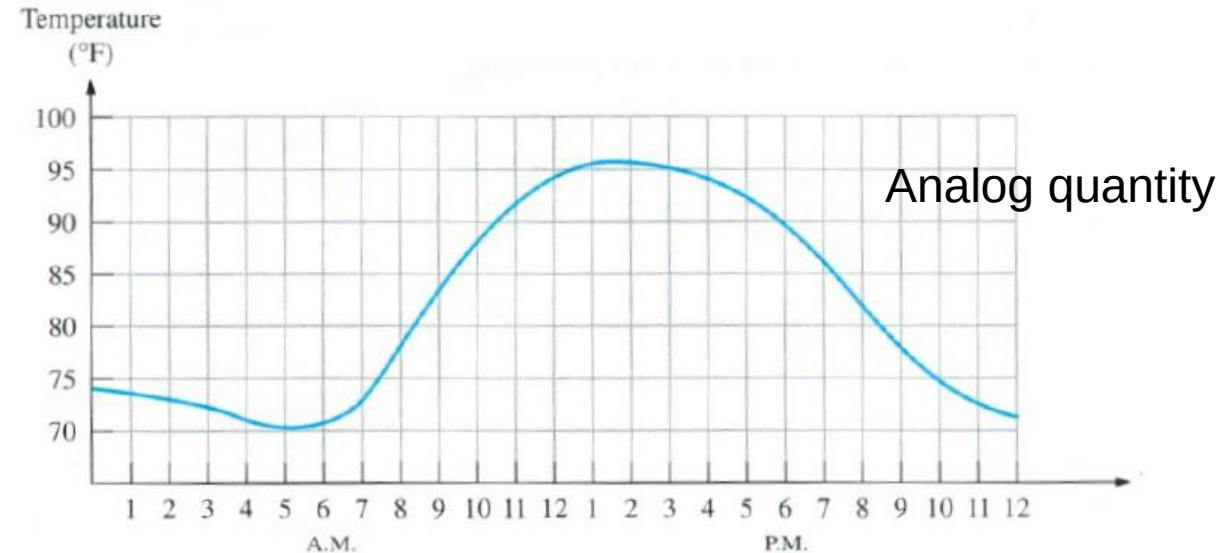


- **Digital Number Systems**
- Data Representation
- Binary Codes and Logic
- FPGA for Digital Logic

Analog vs. Digital Signals

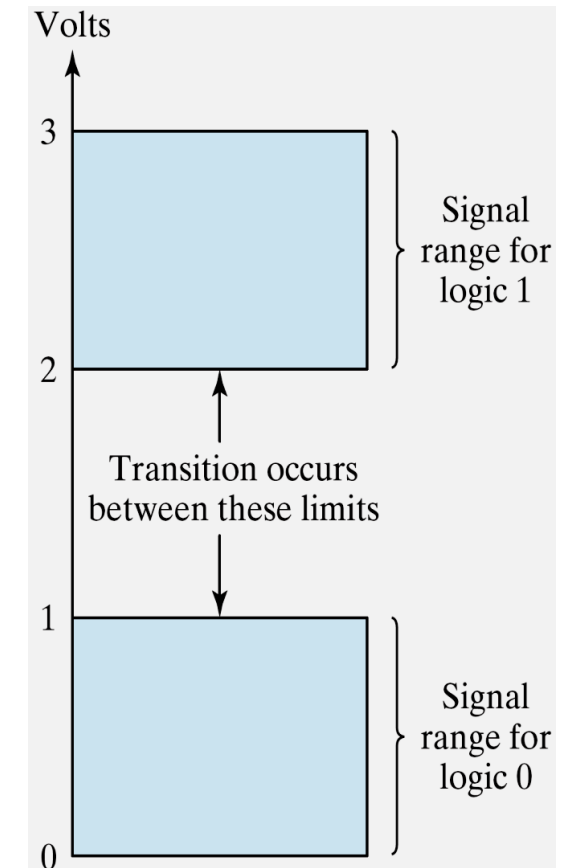
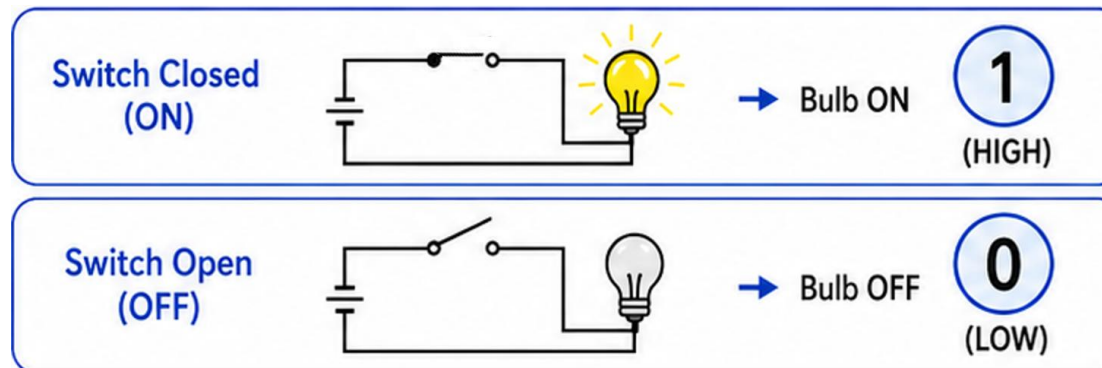


- Signal definition
 - Quantity that can represent and convey information
 - Passed between devices to send and receive information
- Analog signals
 - Converts information into waves of varying amplitude and frequency
 - Continuous waveform
- Digital signals
 - ON (1) or OFF (0) pulses (i.e. binary)
 - Discrete waveform



Binary Digits and Logic Levels

- Bit: binary digit
 - 1: HIGH (TRUE)
 - 0: LOW (FALSE)
- Codes: group of bits (combinations of 1s and 0s)
 - Used to represent numbers, letters, symbols, instructions, and anything else required in a given application
- Logic levels

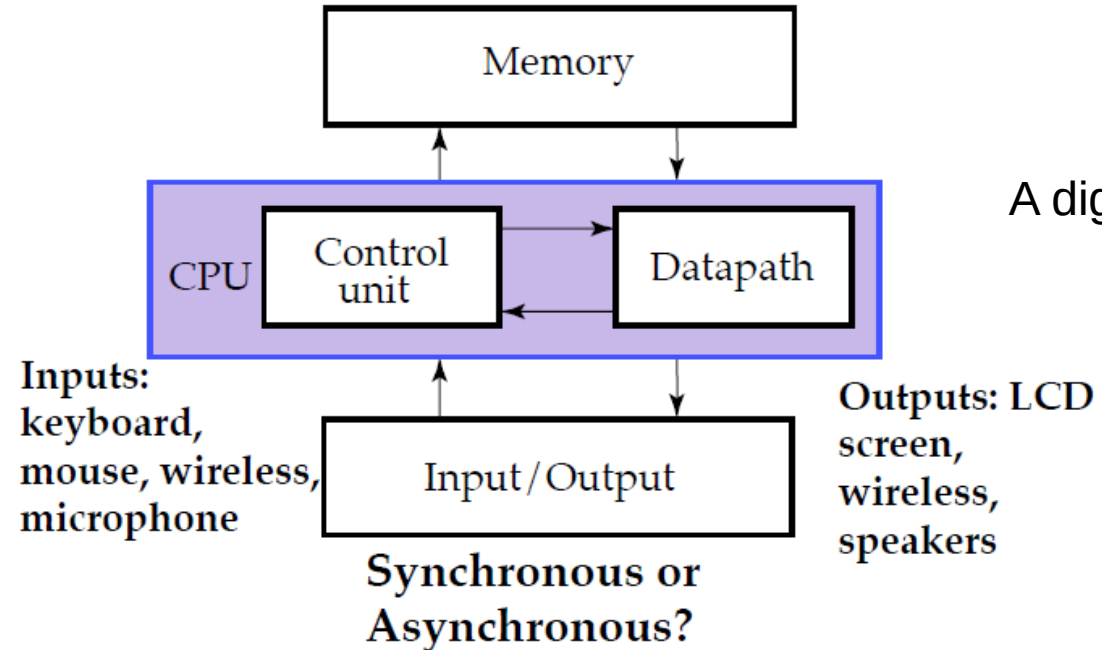


Digital Systems



- A digital system is a system that processes digital signals or data. It operates on discrete values and performs operations such as logic, arithmetic, and data storage in a binary format.
- Digital systems are prevalent in modern electronics, including computers, smartphones, and digital communication devices, due to their reliability and ease of processing.

Understanding digital systems: first step toward more advanced topics like Computer Organization.



A digital computer

Common Number Systems

- It is natural for human to use **decimal system**(十进制)
- In a digital world, we think in **binary**(二进制)
- The **octal** (八进制) and **hexadecimal** (十六进制) numbers are shorter forms for representing binary numbers.

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
0	0000	00	0x0
1	0001	01	0x1
2	0010	02	0x2
3	0011	03	0x3
4	0100	04	0x4
5	0101	05	0x5
6	0110	06	0x6
7	0111	07	0x7
8	1000	010	0x8
9	1001	011	0x9
10	1010	012	0xA
11	1011	013	0xB
12	1100	014	0xC
13	1101	015	0xD
14	1110	016	0xE
15	1111	017	0xF

Outline



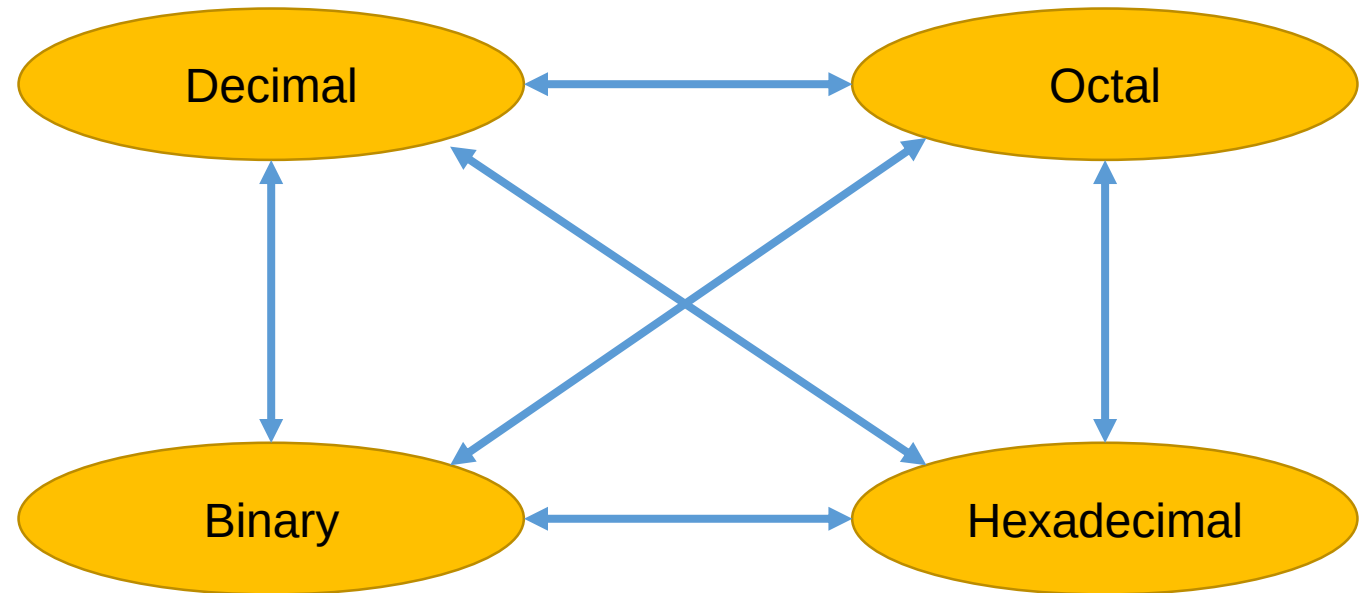
- Digital Number Systems
- **Data Representation**
- Binary Codes and Logic
- FPGA for Digital Logic

Conversion among Bases

- A quick example:
 - $25_{10} = 11001_2 = 31_8 = 19_{16}$

Base or Radix

- The possibilities:



Radix-r to Decimal Conversion

- We use Positional Number Systems: Let r be the **radix** (or **base**), then the $(n+m)$ -digit number

has the value

$$D = d_{n-1}d_{n-2} \dots d_1d_0.d_{-1}d_{-2}\dots d_{-m} \quad 0 \leq d < r$$

Decimal Value radix point

$$D = d_{n-1}r^{n-1} + d_{n-2}r^{n-2} + \dots + d_1r + d_0 + d_{-1}r^{-1} + d_{-2}r^{-2} + \dots + d_{-m}r^{-m}$$

Most-significant Digit (MSD) Least-significant Digit (LSD)

$$D = \sum_{i=-m}^{n-1} d_i r^i$$

Radix-r to Decimal Conversion

- **Decimal** Number System: Base (radix) $r = 10$

2 1 0 -1 -2

5 **1** **2** • **7** **4**

- Coefficients $D=(d_2d_1d_0.d_{-1}d_{-2}) = (512.74)_{10}$
- $(512.74)_{10} = 5 \times 10^2 + 1 \times 10^1 + 2 \times 10^0 + 7 \times 10^{-1} + 4 \times 10^{-2}$

- **Binary** Number System: Base (radix) $r = 2$

2 1 0 -1 -2

1 **0** **1** • **0** **1**

- Coefficients $D=(b_2b_1b_0.b_{-1}b_{-2}) = (101.01)_2$
- $(101.01)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = (5.25)_{10}$

Exercise:

$$1010.101_2 = ?_{10}$$

$$22.22_4 = ?_{10}$$

$$12.5_8 = ?_{10}$$

$$A.A_{16} = ?_{10}$$

Radix-r to Decimal Conversion

$$D = d_{n-1}r^{n-1} + d_{n-2}r^{n-2} + \dots + d_1r + d_0 + d_{-1}r^{-1} + d_{-2}r^{-2} + \dots + d_{-m}r^{-m}$$

Most-significant Digit (MSD)

Least-significant Digit (LSD)

Exercise:

$$1010.101_2 = 1*2^3 + 0*2^2 + 1*2^1 + 0*2^0 + 1*2^{-1} + 0*2^{-2} + 1*2^{-3} = 10.625_{10}$$

$$22.22_4 = 2*4^1 + 2*4^0 + 2*4^{-1} + 2*4^{-2} = 10.625_{10}$$

$$12.5_8 = 1*8^1 + 2*8^0 + 5*8^{-1} = 10.625_{10}$$

$$A.A_{16} = 10*16^0 + 10*16^{-1} = 10.625_{10}$$



Decimal to Radix-r Conversion

- Integer part: Successive divisions by r and observe the remainders
- Fraction: Successive multiplications by r and observe the integer part

Decimal to Binary Conversion (1)

- For Integer part
- Divide the number by the 'Base' (=2)
- Take the remainder (either 0 or 1) as a coefficient
- Take the quotient and repeat the division

Example: $(13)_{10}$

	Quotient	Remainder	Coefficient
$13 / 2 =$	6	1	$a_0 = 1$
$6 / 2 =$	3	0	$a_1 = 0$
$3 / 2 =$	1	1	$a_2 = 1$
$1 / 2 =$	0	1	$a_3 = 1$

Answer: $(13)_{10} = (a_3 a_2 a_1 a_0)_2 = (1101)_2$

MSB LSB

Decimal to Binary Conversion (2)

- For Fraction part, the computation is reversed again
- Multiply the number by the 'Base' (=2)
- Take the integer (either 0 or 1) as a coefficient
- Take the resultant fraction and repeat the division

Example: $(0.625)_{10}$

		Integer	Fraction	Coefficient
0.625	$* 2 =$	1	$. 25$	$a_{-1} = 1$
0.25	$* 2 =$	0	$. 5$	$a_{-2} = 0$
0.5	$* 2 =$	1	$. 0$	$a_{-3} = 1$

Answer: $(0.625)_{10} = (0.a_{-1}a_{-2}a_{-3})_2 = (0.101)_2$

$\uparrow$ $\uparrow$
MSB LSB

Decimal to Binary Conversion (3)

- Easier method

Example: $(35.375)_{10}$

2	35		
2	17	...	1
2	8	...	1
2	4	...	0
2	2	...	0
2	1	...	0
	0	...	1

• $(100011.011)_2$

	0.375		
x	2		
	0.750	...	0
x	2		
	1.50	...	1
x	2		
	1.0	...	1

Decimal to Octal Conversion



Example: $(175.3125)_{10}$

	Quotient	Remainder	Coefficient
$175 / 8 =$	21	7	$a_0 = 7$
$21 / 8 =$	2	5	$a_1 = 5$
$2 / 8 =$	0	2	$a_2 = 2$

Integer part: $(175)_{10} = (a_2 a_1 a_0)_8 = (257)_8$

	Integer	Fraction	Coefficient
$0.3125 * 8 =$	2	. 5	$a_{-1} = 2$
$0.5 * 8 =$	4	. 0	$a_{-2} = 4$

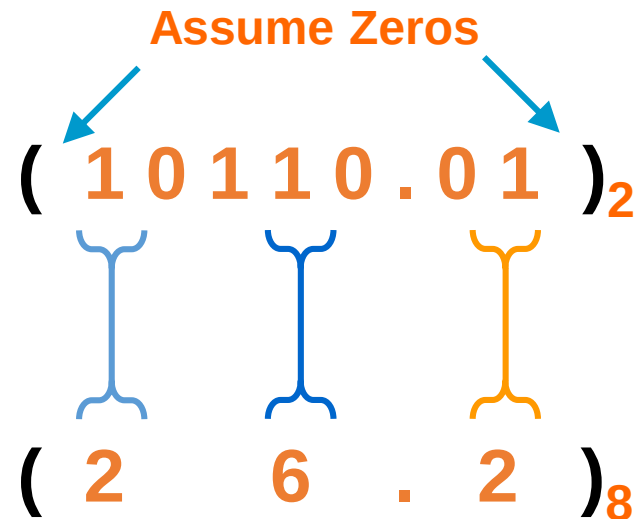
Fraction part: $(0.3125)_{10} = (0.a_{-1} a_{-2} a_{-3})_8 = (0.24)_8$

Answer: $(175.3125)_{10} = (a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3})_8 = (257.24)_8$

Radix-r to Radix-r Conversion

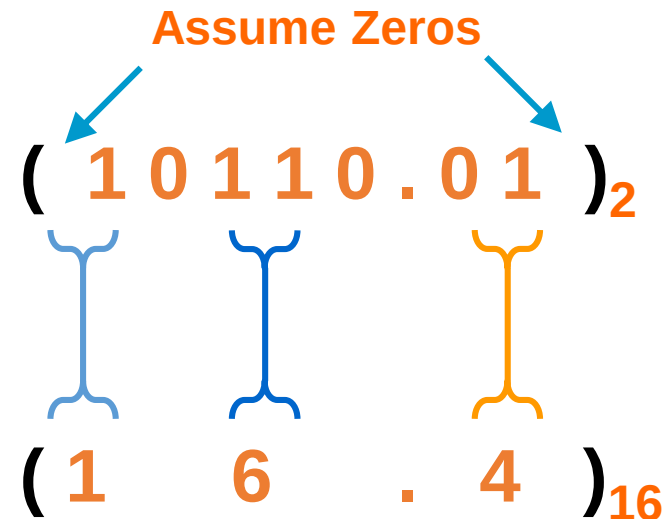
- Binary – Octal

- Each group of 3 bits represents an octal digit starting from radix point
- Works both ways (Binary to Octal & Octal to Binary)



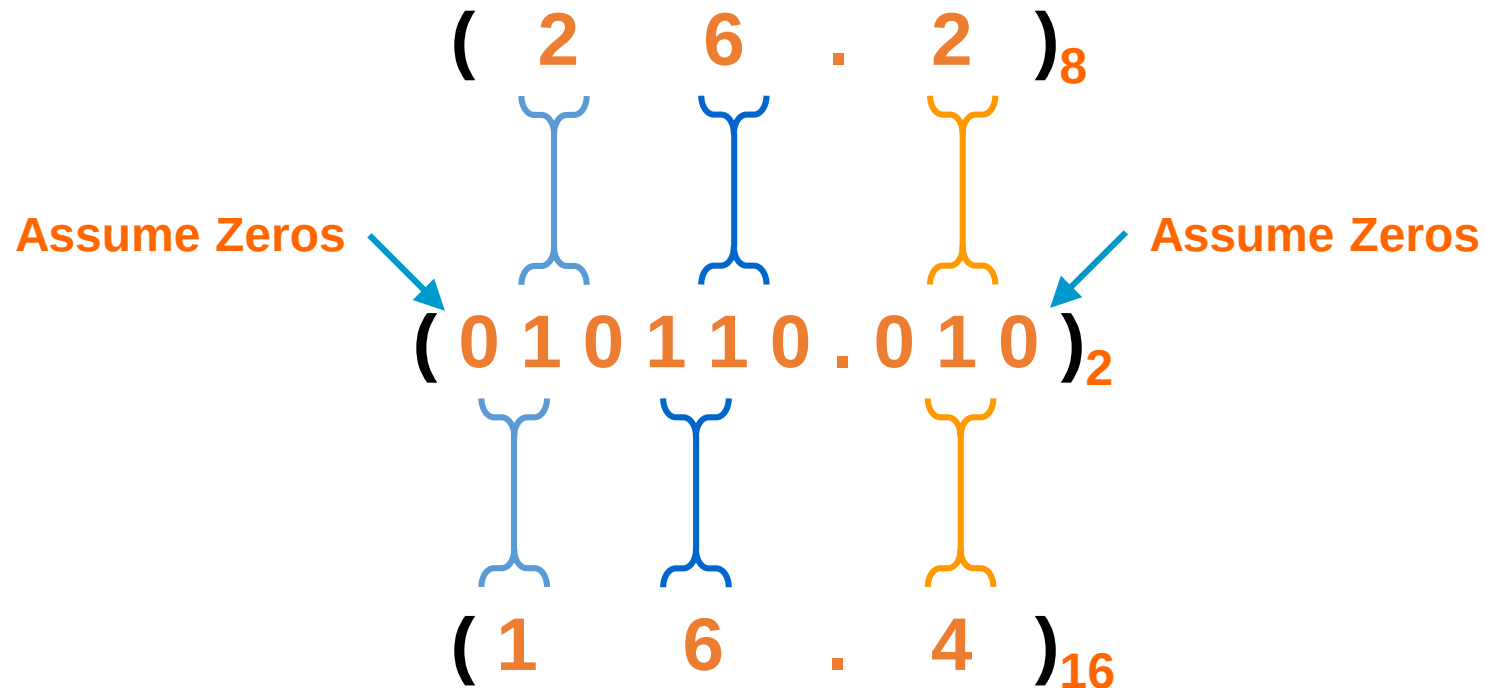
- Binary – Hexadecimal

- Each group of 4 bits represents a hexadecimal digit starting from radix point
- Works both ways (Octal to Hex & Hex to Octal)



Radix-r to Radix-r Conversion (2)

- Octal – Hexadecimal
- Convert to **Binary** as an intermediate step



Common Notions

- Bits(位/位元)

10010110

most significant bit least significant bit

- Bytes(字节)

- 1 byte = 8 bits

byte

10010110

- Hexadecimal representation

- two hexadecimal digit represent one byte value
- often used to represent memory address

0XCEBF9AD7

most significant byte least significant byte

Power	Meaning	Prefix	Symbol
2^{10}	1024	Kilo	K
2^{20}	1024^2	Mega	M
2^{30}	1024^3	Giga	G
2^{40}	1024^4	Tera	T
2^{50}	1024^5	Peta	P
2^{60}	1024^6	Exa	E
2^{70}	1024^7	Zetta	Z

e.g. 1MB = 1024KB

Binary Addition

- Same rules as for decimal numbers
- Column Addition

							carry	
	1	1	1	1	1	1		
		1	1	1	1	0	1	$(10)_2 = (2)_{10}$
			1	0	1	1	1	= 61 augend
+			1	0	1	1	1	= 23 addend
	1	0	1	0	1	0	0	= 84 sum

Binary Subtraction

- Same rules as for decimal numbers
- Borrow a “Base” when needed

					borrow		
		1		2			= (10) ₂
	0	2	2	0	0	2	
	1	0	0	1	1	0	= 77 minuend
-			1	0	1	1	= 23 subtrahend
	0	1	1	0	1	1	= 54 difference

Complements

- When human do subtraction, we use “borrow” concept to borrow a 1 from a higher significant position.
- It is hard for circuits to design “borrow”. So complements are used to implement subtraction.
 - Simplify the subtraction operation.
 - Simpler, less expensive circuits.
- Two types for radix-r system
 - Diminished radix complement (反码) $((r-1)$'s-complement)
 - Radix complement (补码) $(r$'s-complement)
- Examples:
 - For a binary system: 1's complement and 2's complement.
 - For a decimal system: 9's complement and 10's complement.
- Question:
 - For a radix-7 system: ?

Complements for decimal system

- Diminished radix complement
 - $r^n - 1 - x$, assuming r is the radix, n is number of bits, x is the value
 - 9's-complement of 540 = $999 - 540 = 459$
 - 9's-complement of 12 = $999 - 012 = 987$ (is it equal to 87?)
- Radix complement
 - $r^n - x$, assuming r is the radix, n is number of bits, x is the value
 - 10's-complement of 540 = $1000 - 540 = 460$
 - 10's-complement of 12 = $1000 - 012 = 988$ (is it equal to 88?)
 - **Easier method 1:** Calculate the diminished radix complement, then plus one
 - 10's-complement of 540 = $999 - 540 + 1 = 460$
 - **Easier method 2:** use r minus the least significant non-zero digit, and $r - 1$ minus digits on the left
 - The least significant non-zero digit of 540 is 4: $10 - 4 = 6$;
 - Digits on the left is 5: $9 - 5 = 4$;
 - The 10's complement of 540 is 460.

Complements for binary system

- 1's Complement (*Diminished Radix Complement*) for binary

- All '0's become '1's
- All '1's become '0's
- just like a bitwise not(按位取反)

Example $(10110000)_2$

$\Rightarrow (01001111)_2$

- 2's Complement: 1's complement, then plus one:

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\
 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1 \\
 + \qquad \qquad \qquad 1 \\
 \hline
 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0
 \end{array}$$

Another way to find 2's complement: leave the first non-zero LSB unchanged, and then replacing 1's with 0's and 0's with 1's in the other MSBs:

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\
 \quad \quad \quad \leftarrow \text{first non-zero LSB} \\
 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0
 \end{array}$$

Subtraction with Complements

- Algorithm:
- Subtraction of two k-digit unsigned numbers $M - N$
 - Replace subtraction with addition
 - M and N **both** have k-digit
- $M - N = M + r$'s complement of N
 - If $M \geq N$, the sum will produce an end **carry** r^k which is **discarded**, and what is left is the result $M - N$
 - If $M < N$, the sum does not produce an end carry. It is equal to the r's complement of $(M - N)$. The correct answer is generated by
 1. taking the r's **complement** of the answer
 2. then adding a **negative sign** to the front
- Pay attention to align the number of digits for two operands

Subtraction with 10's Complement

- Example with $M \geq N$
 - Using 10's complement, subtract $72532 - 3250$.
- Example with $M < N$
 - Using 10's complement, subtract $3250 - 72532$.

$$\begin{array}{r}
 M = 72532 \\
 10's \text{ complement of } N = +96750 \\
 \hline
 \text{Sum} = 169282 \\
 \text{Discard end carry } 10^5 = -100000 \\
 \hline
 \text{Answer} = 69282
 \end{array}$$

$$\begin{array}{r}
 M = 03250 \\
 10's \text{ complement of } N = +27468 \\
 \hline
 \text{Sum} = 30718
 \end{array}$$



There is **no end carry**.
 Answer = $-(10's \text{ complement of } 30718)$
 $= -69282$.

Note: When working with paper and pencil, we can change the answer to a signed negative number in order to put it in a familiar form.

Subtraction with 2's Complement

- Example:
 - Given the two binary numbers $X = 1010100$ and $Y = 1000011$ ($X > Y$), perform the subtraction (a) $X - Y$; and (b) $Y - X$, by using 2's complement.

$$\begin{array}{rcl} \text{(a)} & X = & 1010100 \\ & 2\text{'s complement of } Y = & \underline{+0111101} \\ & \text{Sum} = & 10010001 \\ & \text{Discard end carry } 2^7 = & \underline{-10000000} \\ & \text{Answer. } X - Y = & 0010001 \end{array}$$

$$\begin{array}{rcl} \text{(b)} & Y = & 1000011 \\ & 2\text{'s complement of } X = & \underline{+0101100} \\ & \text{Sum} = & 1101111 \end{array}$$



There is no end carry.
Answer. $Y - X$
 $= - (2\text{'s complement of } 1101111)$
 $= - 0010001.$

Signed Binary Numbers

- To represent Signed numbers in real life:
 - We have + and –
- To represent signed number in digital system
 - everything in binary digit
- **Three types** of representations of signed binary numbers:
 - Sign-magnitude representation
 - Signed-1's complement representation
 - Signed-2's complement representation
- In Signed binary system, the convention is to make the **sign bit (MSB)**
 - **0 for positive**
 - and **1 for negative**

3 Types of Signed Binary Numbers

- Example, assume 9-bits number representation:

- $(105)_{10}$?

- $105_{10} = 1101001_2$, represent in 9 bits

- Signed-magnitude representation of 105: **0**01101001
- Signed-1's-complement representation of 105: **0**01101001
- Signed-2's-complement representation of 105: **0**01101001

MSb 8bits

S	7	6	5	4	3	2	1	0
0	0	1	1	0	1	0	0	1
0	0	1	1	0	1	0	0	1
0	0	1	1	0	1	0	0	1

- $(-105)_{10}$?

- Magnitude of -105 is 1101001, represent in 9 bits

- Signed-magnitude representation of -105: **1**01101001
- Signed-1's-complement representation of -105: **1**10010110
- Signed-2's-complement representation of -105: **1**10010111

MSb 8bits

S	7	6	5	4	3	2	1	0
1	0	1	1	0	1	0	0	1
1	1	0	0	1	0	1	1	0
1	1	0	0	1	0	1	1	1

Which Signed Binary Number to Choose?

- All possible 4-bit signed binary numbers in the three representations.

- Which one is the best? Why?

- Noted:

- To unified representation,
- for positive number, the complement is the same as the original form
- for negative number, the complement is obtained by finding the 1's complement of its absolute value representation, then add 1

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

Which Signed Binary Number to Choose?

	Addition	Representation of 0	Range
Sign-magnitude	Doesn't work $\rightarrow -6+6$ $\begin{array}{r} 1110 \\ + 0110 \\ \hline 10100 \text{ (wrong!)} \end{array}$	Two representations $0000 \text{ } +0$ $1000 \text{ } -0$	$[-(2^{N-1}-1), 2^{N-1}-1]$
Signed-1's complement	Doesn't work $\rightarrow -3+6$ $\begin{array}{r} 1100 \\ + 0110 \\ \hline 10010 \text{ (wrong!)} \end{array}$	Two representations $0000 \text{ } +0$ $1111 \text{ } -0$	$[-(2^{N-1}-1), 2^{N-1}-1]$
Signed-2's complement	Works $\rightarrow -3+6$ $\begin{array}{r} 1101 \\ + 0110 \\ \hline 10011 \text{ (correct!)} \end{array}$	Only one $0000 \text{ } \pm 0$ $1000 \text{ is } -8$	$[-2^{N-1}, 2^{N-1}-1]$

Outline



- Digital Number Systems
- Data Representation
- **Binary Codes and Logic**
- FPGA for Digital Logic

BCD Codes



- BCD Code
 - Four bits are required to code each decimal number.
 - Example:
 - Decimal **396** is represented in BCD with 12bits as **0011 1001 0110**, with each group of 4 bits representing one decimal digit.
 - Also known as 8-4-2-1 code, as 8, 4, 2, and 1 are the weights of the four bits of BCD.
- Notes:
 - The essence is converting decimal numbers “digit by digit” into binary, rather than converting the entire number as a whole
 - The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

Decimal Symbol	BCD Digit	Decimal Symbol	BCD Digit
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

BCD Addition

- First add the two numbers using normal rules for binary addition.
- If the 4-bit sum is equal to or less than 9, it becomes a **valid** BCD number.
- If the 4-bit sum is **greater than 9**, or if a **carry** out of the group is generated, it is an **invalid** result.
 - In such a case, **add $(0110)_2$ or $(6)_{10}$** to the 4-bit sum in order to skip the six invalid states and return the code to BCD. If a carry results when 6 is added, add the carry to the next 4-bit group.
- Example: Consider the addition of $184 + 576 = 760$ in BCD:

BCD		1	1	
	0001	1000	0100	184
	+ 0101	0111	0110	+576
Binary sum	0111	10000	1010	
Add 6		0110	0110	
BCD sum	0111	0110	10000	760

Annotations:

- Red arrows point from the carry '1' above the second and third 4-bit groups to the '1' in the 'BCD sum' row for those groups.
- Green text 'valid result' is under the first 4-bit sum (0111).
- Green text 'greater than 9' is next to the third 4-bit sum (1010).
- Red circles highlight the '1' in the second 4-bit sum (10000) and the '1' in the third 4-bit sum (10000).

Question: Why adding 6 if greater than 9 or carry occurs?

BCD Subtraction

- Same as in the binary case:
- Take the **10's complement** of the subtrahend and add it to the minuend.
- Example: Consider the subtraction of $109 - 132 = -23$ in BCD:
 - Take 10's comp of 132 = 868
 - Convert difference into 10's complement

Subtraction		1		
	0001	0000	1001	109
	+1000	0110	1000	+868
Binary sum	1001	0111	1 0001	
Add 6	0000	0000	0110	
Difference	1001	0111	0111	977
10's complement				-23

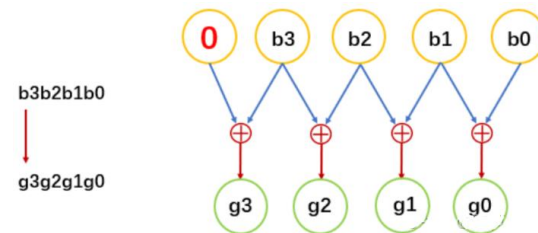
Gray Code

- Gray Code(格雷码)

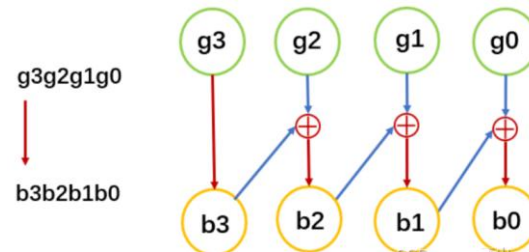
- Minimum change code: A number changes by only one bit as it proceeds from one number to the next.
 - Error detection.
 - Representation of analog data.
 - Low power design.

- Gray & Binary Conversion

- $G_i = B_i \oplus B_{i+1}$



- $B_n = G_n, B_{i-1} = B_i \oplus G_{i-1}$



Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

ASCII Codes

- American Standard Code for Information Interchange (ASCII) Character Code
 - Many applications of the computer require not only handling of numbers, but also of letters.
 - To represent letters it is necessary to have a binary code for the alphabet.
 - Seven bits to code 128 characters.

$b_4b_3b_2b_1$	$b_7b_6b_5$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	—	o	DEL

Error-Detecting Code

- Error-Detecting Code (奇偶校验码)
 - To detect errors in data communication and processing, an extra (eighth in this example) bit is sometimes added to the ASCII character to indicate its parity.
 - A **parity bit** (校验位) is an extra bit included with a message to make the total number of 1's either even or odd.

- Example:

	With even parity	With odd parity
ASCII A = 1000001	0 1000001	1 1000001
ASCII T = 1010100	1 1010100	0 1010100

- Suppose we use even parity

Original code	With even parity	sender	receiver	Parity check Passed?
1000001	01000001	01000001	01000001	yes
1000001	01000001	0100 0 001	0100 1 001	No
1000001	01000001	0100 00 01	0100 11 01	Yes but fails for double errors

limitation, unable to check multiple bits error



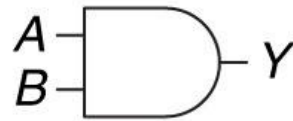
Binary Logic

- Binary logic deals with binary variables(e.g. can have two values,“0”and “1”)
- Binary variables can undergo three basic logical operators AND, OR and NOT
 - **AND** is denoted by a dot ($\cdot$) $z = x \cdot y$ or $z = xy$.
 - **OR** is denoted by a plus ($+$) $z = x + y$.
 - **NOT** is denoted by a single quote mark (') after the variable, or an overbar ($\bar{\quad}$) above the variable.
 - $x'y$ is pronounced as "x prime y" or "x complement y".
- Binary logic resembles binary arithmetic.
 - However, binary logic should not be confused with binary arithmetic.
 - An arithmetic variable designates a number that may consist of many digits.
 - A logic variable is always either 0 or 1.

Binary Logic

- Truth Tables, Boolean Expressions, and Logic Gates

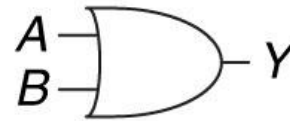
AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

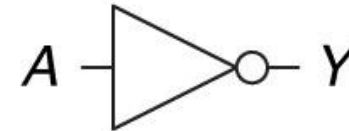
OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

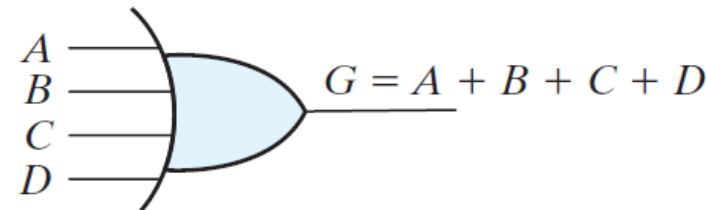
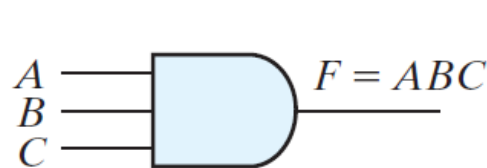
NOT



$$Y = \bar{A}$$

A	Y
0	1
1	0

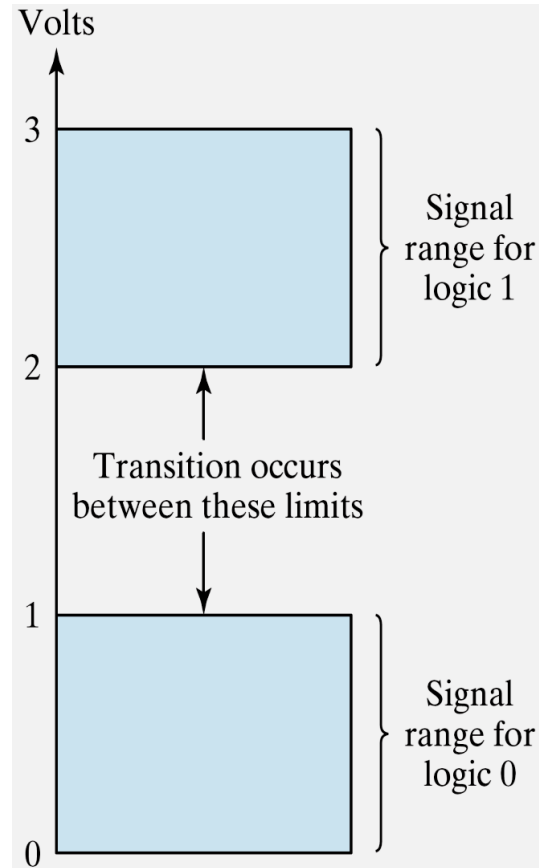
- It is fine to have more than two inputs for AND/OR



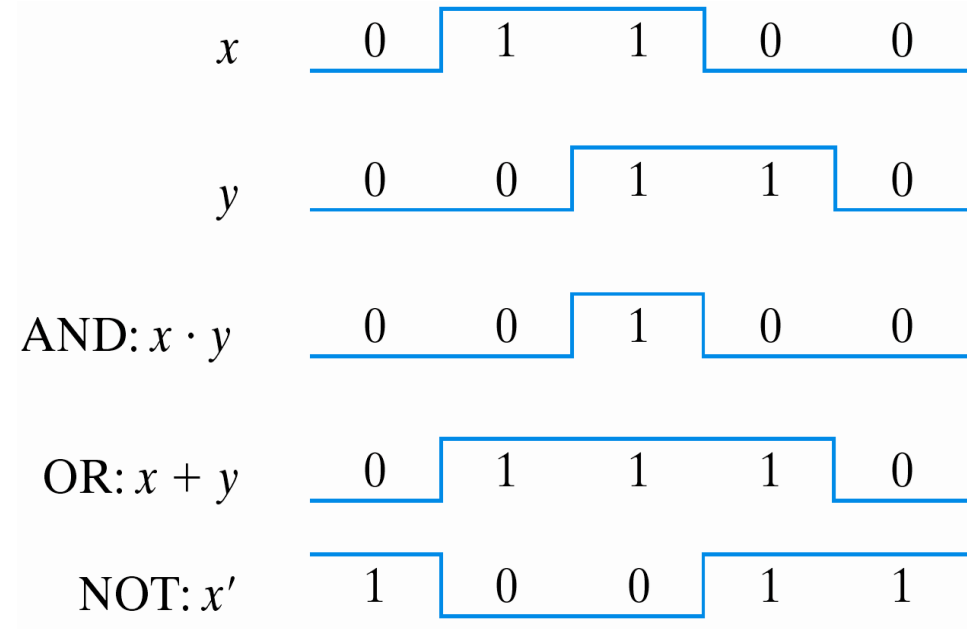
Binary Logic



- Voltage-operated, though on a range, interpreted to be either of the two values



Signal levels for binary logic values



Input–output signals for gates

Summary



- Number Systems
 - Binary, Octal, Hexadecimal
 - Conversion: Use weighted sums (to Decimal) and division/multiplication (from Decimal).
- Binary Arithmetic
 - Addition/Subtraction: Column-wise with carry/borrow.
 - Complements: Simplify subtraction.
 - 1's Comp: Bitwise NOT.
 - 2's Comp: (1's Comp + 1); preferred method.
- Signed Numbers
 - MSB is Sign Bit: 0 = Positive, 1 = Negative.
 - 2's Complement is Standard: Uniquely represents zero, simplifies arithmetic.
- Binary Codes
 - BCD: 4 bits per decimal digit.
 - Gray Code: Adjacent values differ by 1 bit.
 - Parity Bit: Single-bit error detection.
- Binary Logic
 - Gates (AND, OR, NOT): Building blocks.
 - Logic Levels: HIGH (1) / LOW (0) voltage ranges.

Outline



- Digital Number Systems
- Data Representation
- Binary Codes and Logic
- **FPGA for Digital Logic**

Calculate $a + b$ using CPU

- How to calculate $a + b$?

```
int adder(int a, int b)
{
    int z = a + b;
    return z;
}
```

C Programming language

Compilation



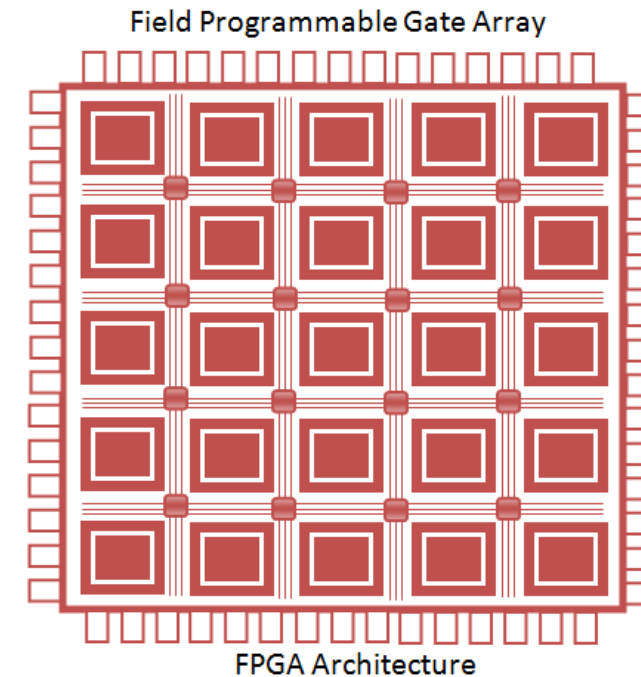
Calculate $a + b$ using FPGA

- How to calculate $a + b$?

```
module adder(  
    input wire [4:0] a,  
    input wire [4:0] b,  
    output wire [4:0] z  
);  
    assign z = a + b;  
endmodule
```

Hardware Description Language (HDL)

Synthesis

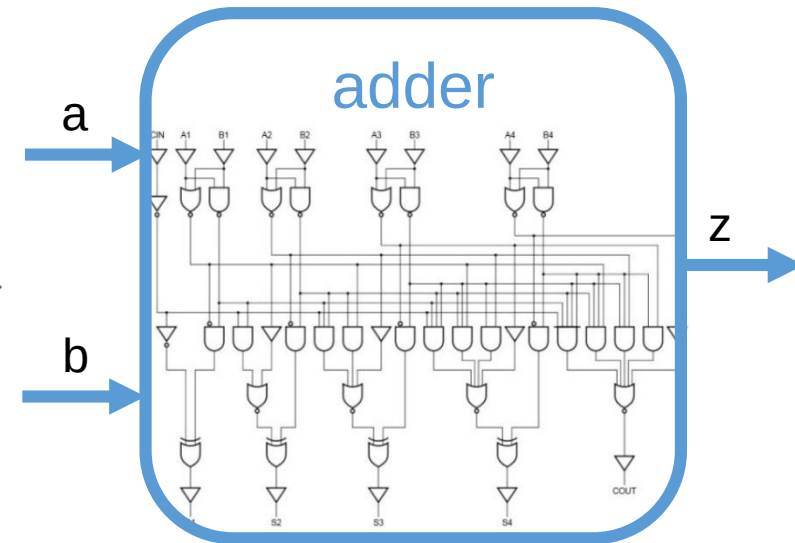


Hardware design

- These hardware blocks are comprised completely of registers and logic gates

```
module adder(  
    input wire [4:0] a,  
    input wire [4:0] b,  
    output wire [4:0] z  
);  
    assign z = a + b;  
endmodule
```

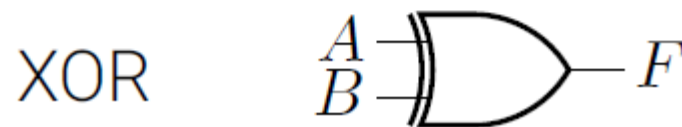
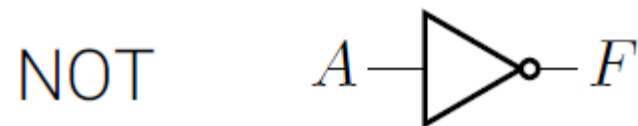
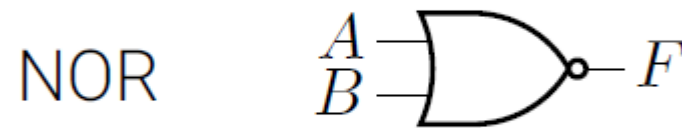
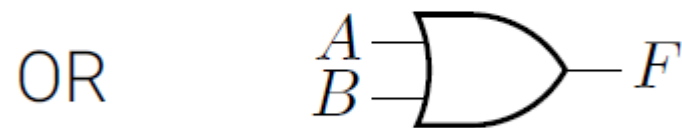
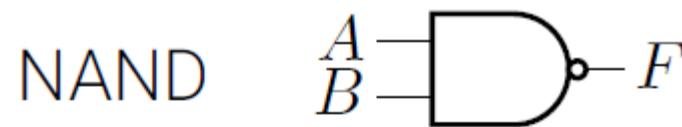
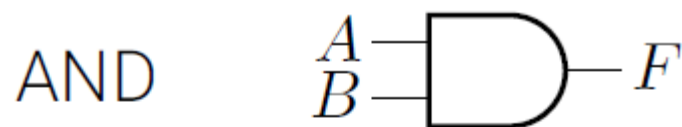
Synthesis



Hardware Description Language (HDL)

Hardware Schematic

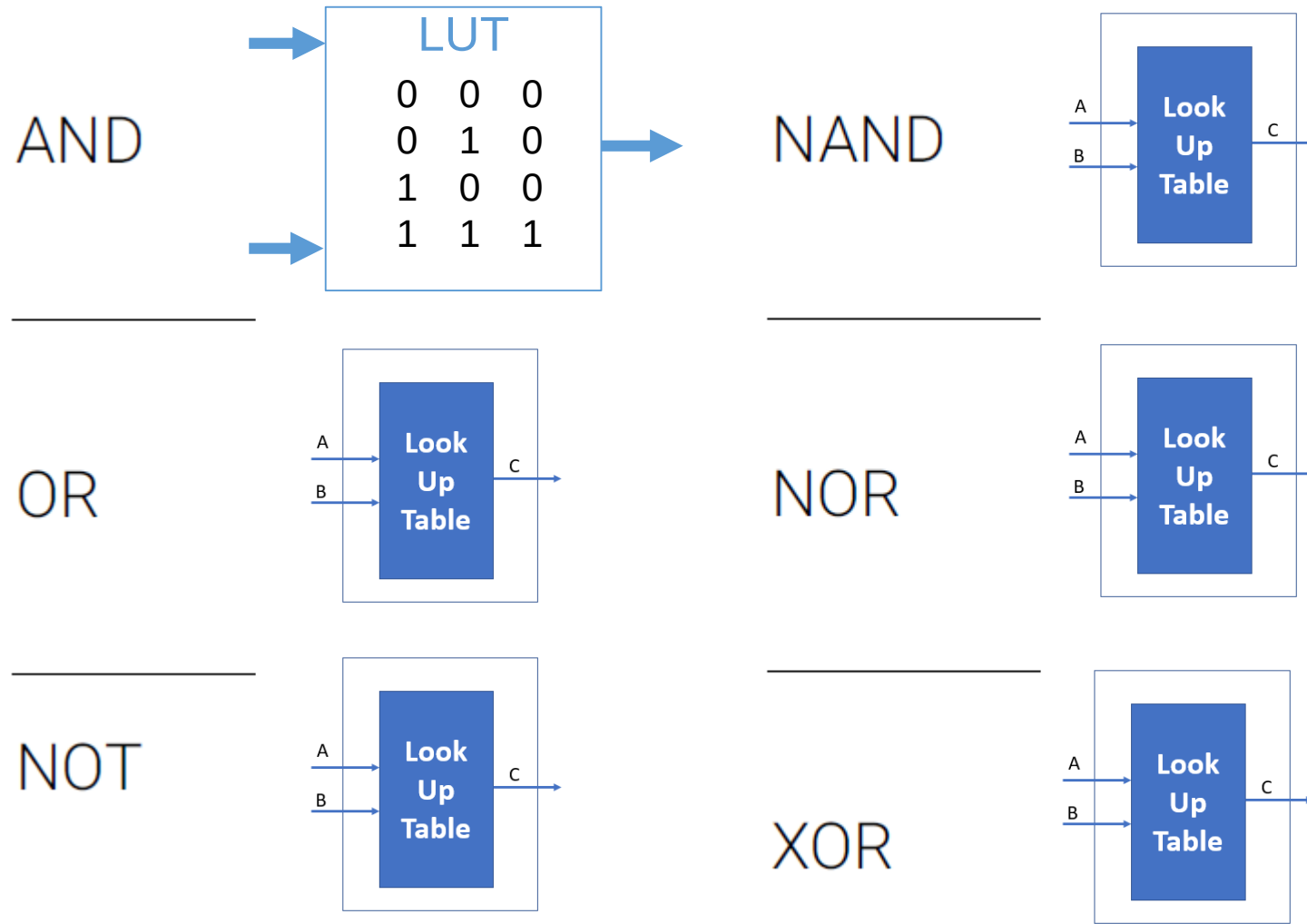
Logic gates



Logic gates



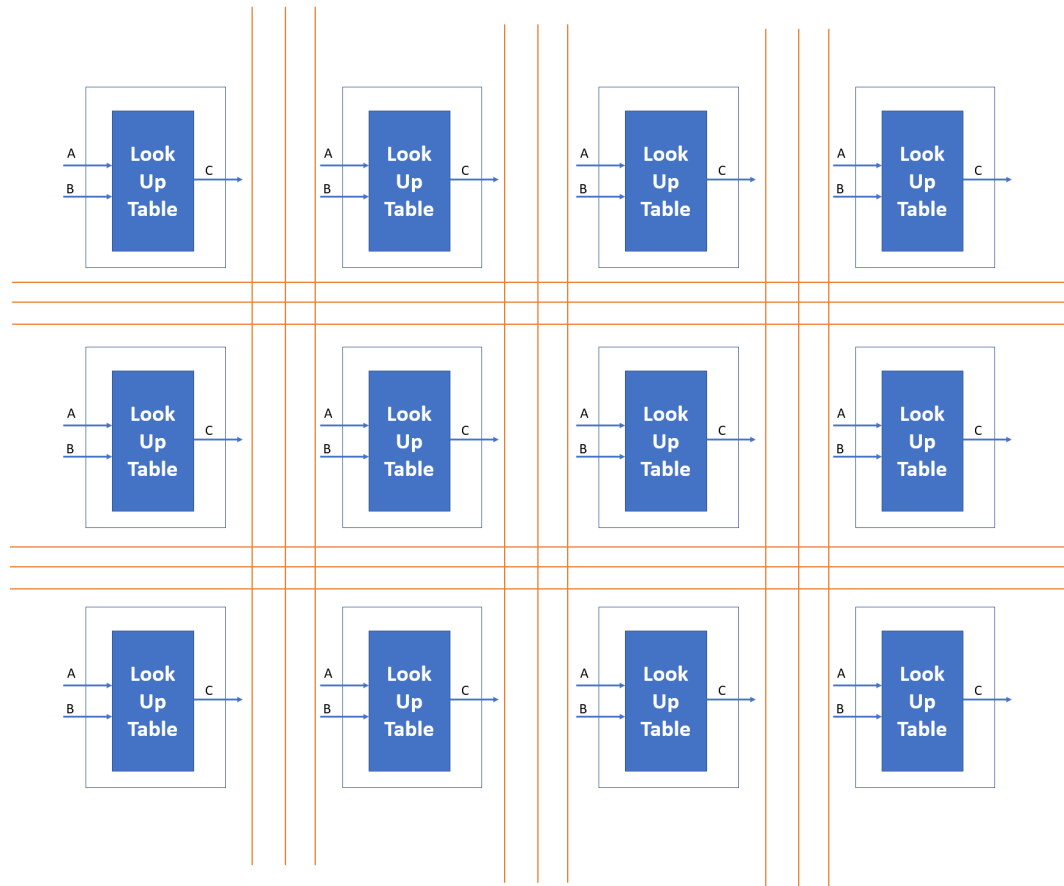
- The logic gates can be implemented using look-up tables.



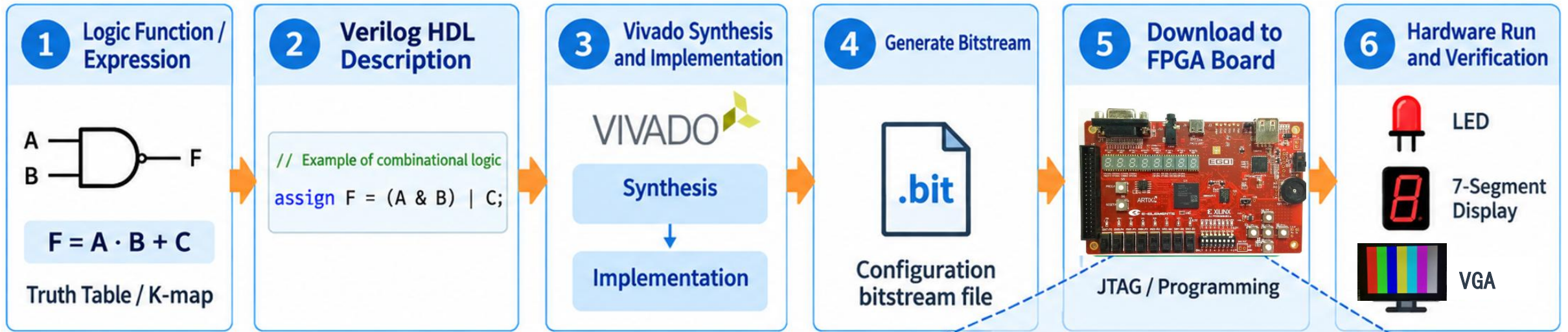
Programmable FPGA



- If you put together a bunch of look-up tables, and make them programmable, then you add a switching fabric that can connect them all together, it's just like playing with LEGO bricks !



Theory to Practice



- Lab website
- <https://cs207digitallogic.github.io/site/>

